Bayesian logistic regression with the Laplace approximation

Modeling, derivations, and computational examples

Start from the sigmoid model for a binary label. Add a Gaussian prior, find the posterior mode, and approximate the coefficient posterior by a Gaussian. Finally, integrate that Gaussian to predict a new label.

$$p(y_i = 1 \mid x_i, \theta) = \sigma(x_i^\top \theta), \qquad \sigma(a) = \frac{1}{1 + e^{-a}}.$$

$$q(\theta) = \mathcal{N}(\theta_{\text{MAP}}, S_n), \qquad S_n = \left[\nabla^2 J(\theta_{\text{MAP}})\right]^{-1}.$$

$$r_* = \mathbb{E}_q[\sigma(x_*^\top \theta)], \qquad y_* \mid x_*, \mathcal{D} \approx \text{Bernoulli}(r_*).$$

The central demonstration

Exercise 2 computes a Laplace approximation from data, then compares its density with a numerically normalized one-coefficient posterior. Students see a Gaussian that follows the posterior closely before using the same calculation for the intercept-and-slope model.

Companion notebooks

`bayesian_logistic_regression_student.ipynb`

`bayesian_logistic_regression_solutions.ipynb`

The main lesson follows Sections 1–6. It contains three coding exercises: the posterior objective and derivatives, computation of the Laplace approximation, and predictive Monte Carlo averaging. The final example shows a poorly learned linear slope and the effect of adding observations over a wider input range.

The teaching plan is on p. 2; the board formula sheet is on p. 22. The exercise solutions and complete derivations appear in notebook order. Optional numerical linear algebra, evidence, calibration, and OOD material stay outside the timed lesson.

The 90-minute teaching plan

Preparation / exercise time is included in every block

Clock	Topic and notebook location	Student activity	Guide
0–5	1.1–1.2: sigmoid model and dataset	Discuss the modeled quantity	p. 4
5–17	1.3: likelihood, gradient, Hessian	Oral derivative and sign checks	p. 5
17–31	2: Gaussian prior and MAP	Exercise 1: 8 minutes	pp. 6–7
31–38	3: Newton optimization	Inspect convergence	p. 8
38–53	4: Laplace calculation and density plot	Exercise 2: 6 minutes	pp. 9–11
53–62	5.1: predictive distribution	Run the two-logit example	p. 12
62–75	5.2–5.3: quadrature and Monte Carlo	Exercise 3: 6 minutes	pp. 13–14
75–80	6.1–6.3: predictive examples	Interpret curves and intervals	pp. 15–17
80–88	6.4: weak slope and wider-coverage refit	Discuss the uncertainty plots	pp. 18–20
88–90	Complete the prediction story	Brief oral synthesis	p. 21

Before the seminar

Run setup and the main solutions notebook. Project the student notebook and keep this guide and the executed solution privately available. Dependencies for the main lesson are NumPy, SciPy, Matplotlib, and IPython. No network access or image dataset is needed. Students should be able to differentiate scalar functions, multiply matrices, use Bayes' rule, and recognize a Gaussian mean and covariance.

During the seminar

Follow the cells in order. The numerical helpers and plots are supplied; run them without reading every line aloud. Only the three coding exercises require students to enter answers. Other questions are optional oral prompts. When students work in pairs, switch the typing role between exercises.

Protect the predictive endpoint

Start Laplace by minute 38 and prediction by minute 53. If an implementation remains unfinished at the end of its block, explain the missing step and continue with the working solution. Keep the Laplace density overlay, predictive averaging, and the weak-slope comparison; shorten solver details and the Monte Carlo convergence discussion first.

The pages contain supporting explanations for preparation as well as the live narrative. The three predictive figures at minutes 75–80 are already computed; show their interpretation, not their plotting code. Do not turn the uncertainty example into a separate information-theory lecture.

Board layout and notation

Preparation / one sign convention throughout

Keep the model and symbols on the left of the board, derive the current result in the middle, and leave the inference chain on the right. Erase intermediate algebra, not the definitions of the objective and covariance.

Symbol	Meaning and shape
X, x_i	$X \in \mathbb{R}^{n \times d}$ has rows $x_i^\top$; d includes the intercept.
y, p	Binary labels and sigmoid probabilities, both length n.
θ, m_0	Coefficients and prior mean, both length d.
S_0, P_0	Prior covariance; $P_0 = S_0^{-1}$ abbreviates its inverse.
J, g, H	Summed negative log posterior up to a constant, gradient, and Hessian.
W	$\mathrm{diag}(p_i(1 - p_i))$, used symbolically; not allocated in code.
H_{MAP}, S_n	Hessian evaluated at the posterior mode; $S_n = H_{\mathrm{MAP}}^{-1}$ is the Laplace covariance.
m_*, v_*	Mean and variance of the Gaussian logit $a_* = x_*^\top \theta$.
π_*, r_*	Random probability $\pi_* = \sigma(a_*)$ and its posterior average $r_* = \mathbb{E}_q \pi_*$.
S, J_H	Number of Monte Carlo coefficient draws and number of quadrature nodes.

The objective determines the signs

The notebooks minimize $J(\theta) = -\log p(\theta \mid \mathcal{D}) + C$. Thus $H = \nabla^2 J$ is positive at the minimum, the Newton direction solves $H\delta = -g$, and the Laplace covariance is $[\nabla^2 J(\theta_{\mathrm{MAP}})]^{-1}$. A Hessian of the log posterior itself would have the opposite sign.

The meaning of the inverse

For one coefficient, the Gaussian has variance $1/J''(\theta_{\mathrm{MAP}})$. Large curvature makes the density narrow; small curvature makes it broad. In several dimensions, the covariance is the matrix inverse of the Hessian, not the elementwise reciprocal. This is the only inverse-curvature interpretation needed for the main lesson.

Keep the two datasets distinct

The main simulated regression dataset has 24 observations and two coefficients. The small Laplace density demonstration has 40 labels and one coefficient. It is introduced only to show the entire coefficient posterior in one plot. Section 4.3 immediately returns to the original two-coefficient data.

1. The sigmoid model

Notebook Sections 1.1–1.2

Say. "A linear score can be any real number, but a Bernoulli probability must lie between zero and one. Logistic regression makes the log odds linear and uses the sigmoid to recover the probability."

Write and derive

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$, with $y_i \in \{0, 1\}$. Write

$$a_i = x_i^\top \theta, \qquad \log \frac{p_i}{1 - p_i} = a_i.$$

Exponentiate and rearrange:

$$p_i = e^{a_i}(1 - p_i), \qquad p_i = \frac{e^{a_i}}{1 + e^{a_i}} = \sigma(a_i).$$

For one observed label,

$$p(y_i \mid x_i, \theta) = p_i^{y_i}(1 - p_i)^{1 - y_i}.$$

The sigmoid is the conditional response function. It is not a posterior over θ. We condition on the covariates X; no probability model for their distribution is required here.

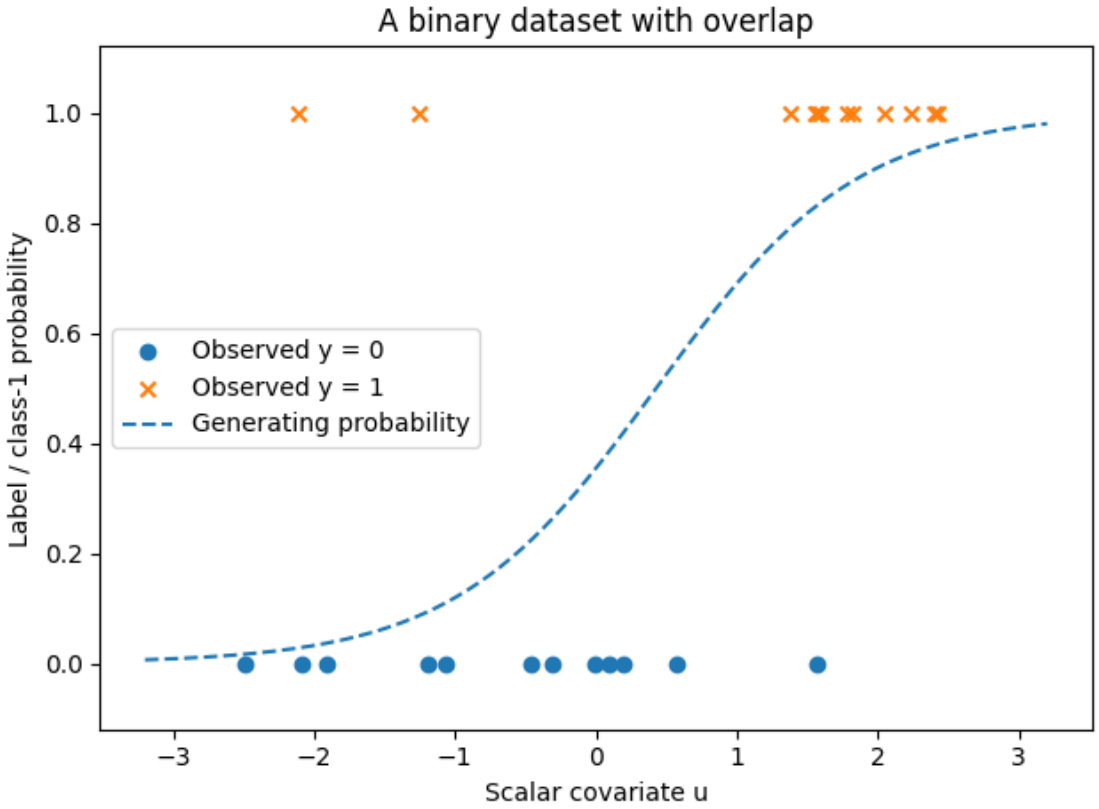


Figure 1: The main dataset has 24 labels, including 12 positive labels. The generating probability is shown only because the data are simulated.

Use $x_i = (1, u_i)^\top$ so that θ_0 is an intercept and θ_1 a slope. The data are generated with $(-0.6, 1.4)^\top$, but that vector is not supplied to inference.

Notebook. The helper `sigmoid(a)` calls `expit(a)`. It accepts a scalar, vector, or matrix. Ask how increasing the slope would change the curve before showing a fitted classifier.

2. Likelihood and derivatives

Notebook Section 1.3

Say. "The same derivatives will serve two purposes: optimizing the coefficients and describing the posterior around the optimum. Start with one observation so that every term has a visible origin."

Write and derive

By conditional independence,

$$\ell_{\text{lik}}(\theta) = \sum_i \left[y_i \log \sigma(a_i) + (1 - y_i) \log \sigma(-a_i) \right].$$

Use $\log \sigma(a) = a - \log(1 + e^a)$ and $\log \sigma(-a) = -\log(1 + e^a)$ to obtain

$$\ell_{\text{lik}}(\theta) = \sum_i \left[y_i a_i - \log(1 + e^{a_i}) \right].$$

Differentiate the response and the one-observation log likelihood:

$$\sigma'(a) = \frac{e^{-a}}{(1 + e^{-a})^2} = \sigma(a)[1 - \sigma(a)],$$

$$\frac{d\ell_i}{da_i} = y_i - p_i, \qquad \frac{d^2\ell_i}{da_i^2} = -p_i(1 - p_i).$$

Since $\nabla_\theta a_i = x_i$, the chain rule gives

$$\nabla \ell_{\text{lik}} = \sum_i x_i (y_i - p_i) = X^\top (y - p),$$

$$\nabla^2 \ell_{\text{lik}} = -\sum_i p_i (1 - p_i) x_i x_i^\top = -X^\top W X.$$

For any vector v,

$$v^\top X^\top W X v = \sum_i p_i (1 - p_i)(x_i^\top v)^2 \geq 0.$$

Thus the log likelihood is concave. The Hessian can be singular when the design matrix lacks column rank. The factor $p_i(1 - p_i)$ is largest at $1/2$, but the feature vector also affects each observation's directional contribution.

From algebra to code

For finite logits, evaluate $\sigma(a)$ with `expit` and $\log \sigma(a)$ with `log_expit`; do not calculate `log(sigmoid(a))` at extreme negative logits. Evaluate $\log(1 - \sigma(a))$ as `log_expit(-a)`. These are the same likelihood terms, computed without taking the logarithm of a rounded zero.

Sigmoid and log-sigmoid implementations: SciPy documentation [S].

3. Gaussian prior and MAP

Notebook Section 2

Say. "A prior gives the coefficients a scale before seeing the labels. The Gaussian prior contributes a quadratic term, but the logistic likelihood is not quadratic. The posterior is therefore not generally Gaussian."

Write and derive

Let $\theta \sim \mathcal{N}(m_0, S_0)$, where S_0 is positive definite, and write $P_0 = S_0^{-1}$. Then

$$\log p(\theta) = C - \tfrac{1}{2}(\theta - m_0)^\top P_0(\theta - m_0).$$

Bayes' rule and the normalizing integral are

$$p(\theta \mid \mathcal{D}) = \frac{p(y \mid X, \theta)p(\theta)}{Z}, \qquad Z = \int p(y \mid X, \theta)p(\theta)\, d\theta.$$

Define the objective to be minimized:

$$J(\theta) = -\sum_i \left[y_i \log \sigma(a_i) + (1 - y_i) \log \sigma(-a_i)\right] + \tfrac{1}{2}(\theta - m_0)^\top P_0(\theta - m_0).$$

Change the likelihood signs and differentiate the prior:

$$g(\theta) = X^\top(p - y) + P_0(\theta - m_0), \qquad H(\theta) = X^\top W X + P_0.$$

For $v \neq 0$, $v^\top H(\theta)v \geq v^\top P_0 v > 0$. The objective is strictly convex. Its likelihood contribution is nonnegative, and

$$J(\theta) \geq \tfrac{1}{2}(\theta - m_0)^\top P_0(\theta - m_0) \longrightarrow \infty \quad \text{as } \|\theta\| \longrightarrow \infty.$$

Continuity and growth at infinity give a minimum; strict convexity makes it unique. This is θ_{MAP}. Also, $0 < p(y \mid X, \theta) \leq 1$, so a proper prior gives a finite positive normalizer.

Regularization and scaling

With $m_0 = 0$ and $P_0 = \alpha I$, the penalty is $\alpha\|\theta\|^2/2$. In this specified parameterization, L2-penalized logistic regression is the MAP calculation. The intercept is penalized in these examples.

Keep the summed objective. Dividing all of J by n leaves its minimizer unchanged but divides its Hessian by n; using that Hessian unchanged in Laplace would multiply the covariance by n.

Exercise 1: objective, gradient, Hessian

Notebook function binary_terms

Students translate the three equations on the board into array operations. The scaffold supplies input validation and the logits; the task is the statistical calculation, not the tests.

```
a = X @ theta
delta = theta - m0
p = sigmoid(a)
weights = sigmoid(a) * sigmoid(-a)

nll = -(y * log_expit(a) + (1-y) * log_expit(-a)).sum()
value = nll + 0.5 * delta @ P0 @ delta
gradient = X.T @ (p-y) + P0 @ delta
hessian = X.T @ (weights[:, None] * X) + P0
return float(value), gradient, hessian
```

The returned shapes are a scalar, $(d,)$, and (d, d). The expression `weights[:, None] * X` scales each row of X by its Bernoulli weight. It avoids building an $n \times n$ matrix.

A hand-check at zero coefficients

At $\theta = 0$, all probabilities equal $1/2$. Thus

$$J(0) = n \log 2 + \tfrac{1}{2} m_0^\top P_0 m_0,$$

$$g(0) = X^\top (\tfrac{1}{2}\mathbf{1} - y) - P_0 m_0, \qquad H(0) = \tfrac{1}{4} X^\top X + P_0.$$

For the main data, $m_0 = 0$ and $n = 24$, so $J(0) = 24 \log 2 \approx 16.6355$. The supplied checks also compare derivatives with central finite differences and use logits of magnitude 1000 to check the log-likelihood implementation.

Useful hints

A wrong gradient sign usually comes from differentiating the log likelihood while returning its negative. A missing P_0 in the Hessian omits the prior. A wrong shape usually comes from summing along the wrong axis or multiplying X by a weight vector without adding the singleton column dimension.

Stop after the checks pass. There is no need to explain the finite-difference helper line by line. The next step uses these three returned objects to fit the posterior mode.

4. Newton optimization

Notebook Section 3

Say. "We can evaluate the posterior objective without knowing its normalizing constant. Newton's method uses its current gradient and second derivatives to propose a better coefficient vector."

Write and derive

At the current iterate, write

$$J(\theta + \delta) \approx J(\theta) + g^\top \delta + \tfrac{1}{2}\delta^\top H \delta.$$

Differentiate in the proposed displacement:

$$g + H\delta = 0, \qquad H\delta = -g.$$

The supplied implementation solves this system:

```
direction = np.linalg.solve(hessian, -gradient)
```

Then update $\theta_{\text{new}} = \theta + \eta\delta$. A full step has $\eta = 1$; the solver shortens it when necessary. Since $g^\top \delta = -g^\top H^{-1} g < 0$ for $g \neq 0$, it is a descent direction. The actual objective is not quadratic, so the full step is not guaranteed to decrease it.

What the supplied solver checks

It rejects non-finite values, backtracks until the objective decreases sufficiently, and stops when the largest absolute gradient component is small. Its decrease test is

$$J(\theta + \eta\delta) \leq J(\theta) + 10^{-4}\eta g^\top \delta,$$

with a small allowance for floating-point round-off. A failed iteration limit or line search raises an error rather than returning an unreliable fit.

Main-fit result	Value
MAP intercept	−0.389642
MAP slope	0.961432
Newton updates	5
Final gradient infinity norm	Approximately 1.05×10^{-13}

Transition to Laplace

Newton uses a quadratic at a current iterate to move toward the optimum. Laplace uses a quadratic at the converged optimum to define a probability distribution. A converged optimizer gives the mode; the next calculation adds a Gaussian description of uncertainty around it.

5. Deriving the Laplace approximation

Notebook Section 4.1

Say. "Near its minimum, the negative log posterior looks like a quadratic. Exponentiating the negative of that quadratic gives a Gaussian. We need its center and width, not a new optimization algorithm."

Write and derive

Start with one coefficient. At the posterior mode, $J'(\theta_{\text{MAP}}) = 0$, so

$$J(\theta) \approx J(\theta_{\text{MAP}}) + \tfrac{1}{2} J''(\theta_{\text{MAP}})(\theta - \theta_{\text{MAP}})^2.$$

Since the posterior is proportional to $e^{-J(\theta)}$,

$$p(\theta \mid \mathcal{D}) \approx \text{constant} \times \exp\left[-\frac{J''(\theta_{\text{MAP}})}{2}(\theta - \theta_{\text{MAP}})^2\right].$$

Compare this with the normal density

$$\frac{1}{\sqrt{2\pi s^2}} \exp\left[-\frac{(\theta - \mu)^2}{2s^2}\right].$$

The center is $\mu = \theta_{\text{MAP}}$ and the variance is $s^2 = 1/J''(\theta_{\text{MAP}})$. The normalizing factor follows from the usual one-dimensional Gaussian integral. Nothing about the unknown posterior normalizer is needed to find these two parameters.

The same calculation for a coefficient vector

Let $\Delta = \theta - \theta_{\text{MAP}}$ and $H_{\text{MAP}} = \nabla^2 J(\theta_{\text{MAP}})$. Then

$$J(\theta) \approx J(\theta_{\text{MAP}}) + \tfrac{1}{2} \Delta^\top H_{\text{MAP}} \Delta,$$

$$q(\theta) = \mathcal{N}(\theta_{\text{MAP}}, S_n), \qquad S_n = H_{\text{MAP}}^{-1}, \qquad H_{\text{MAP}} = X^\top W_{\text{MAP}} X + P_0.$$

A Gaussian with covariance S_n has quadratic term $\Delta^\top S_n^{-1} \Delta / 2$ in its negative log density. Matching that term gives the inverse Hessian directly.

What to emphasize

The mode sets the center; the second derivatives set the width. The Gaussian is an approximation to the coefficient posterior, not to the labels. Its mean equals the posterior mode, not necessarily the posterior mean. The Gaussian prior gives positive curvature here. A local match does not make every posterior globally Gaussian.

Exercise 2: compute the approximation

Notebook function laplace_approximation

The function now takes data and a prior, rather than a precomputed matrix. It fits the posterior mode, evaluates the second derivatives there, and returns the mean and covariance of the resulting Gaussian.

```
def laplace_approximation(X, y, m0, P0):
    terms = lambda theta: binary_terms(theta, X, y, m0, P0)
    fit = fit_newton(terms, m0)
    mean = fit["mode"]
    _, _, hessian = terms(mean)
    covariance = np.linalg.inv(hessian)
    return {"mean": mean, "covariance": covariance}
```

The Newton solver is already supplied. The exercise is the sequence

$$\text{data and prior} \longrightarrow \theta_{\mathrm{MAP}} \longrightarrow H_{\mathrm{MAP}} \longrightarrow \mathcal{N}(\theta_{\mathrm{MAP}}, H_{\mathrm{MAP}}^{-1}).$$

The explicit inverse is appropriate for the one- and two-coefficient calculations used here. Students do not need to implement a matrix factorization. Larger-model numerical methods remain appendix material.

A simple test with a known answer

Use an intercept-only model with 20 labels, ten ones and ten zeros, and prior $\theta \sim \mathcal{N}(0, 1)$. Then

$$J'(\theta) = 20\sigma(\theta) - 10 + \theta, \qquad J''(\theta) = 20\sigma(\theta)[1 - \sigma(\theta)] + 1.$$

The posterior mode is zero. At that point, $J''(0) = 20/4+1 = 6$, so the Laplace approximation is $\mathcal{N}(0, 1/6)$. The supplied test checks exactly these values. A second test uses unbalanced labels to ensure that the implementation really fits a nonzero mode rather than returning the starting point.

Hints and expected mistakes

Evaluate the Hessian at the fitted mode, not at the initial coefficients. Return the matrix inverse, not the Hessian itself. For one coefficient, the covariance array contains the variance; its square root is the standard deviation supplied to `norm.pdf`.

After the tests pass, run the density comparison on the next page. The exercise should finish with a visible approximation, not with a discussion of numerical matrix methods.

6. A Gaussian that follows the posterior

Notebook Sections 4.2–4.3

For a direct density comparison, use $p(y_i = 1 \mid \theta) = \sigma(\theta)$, 24 positive labels and 16 negative labels, and prior $\theta \sim \mathcal{N}(0, 2.5^2)$:

$$J(\theta) = -24 \log \sigma(\theta) - 16 \log \sigma(-\theta) + \frac{\theta^2}{2 \cdot 2.5^2}.$$

Consequently,

$$J'(\theta) = 40\sigma(\theta) - 24 + \theta/2.5^2, \qquad J''(\theta) = 40\sigma(\theta)[1 - \sigma(\theta)] + 1/2.5^2.$$

The new exercise function produces

$$q(\theta) = \mathcal{N}\big(0.398822,\ 0.319885^2\big).$$

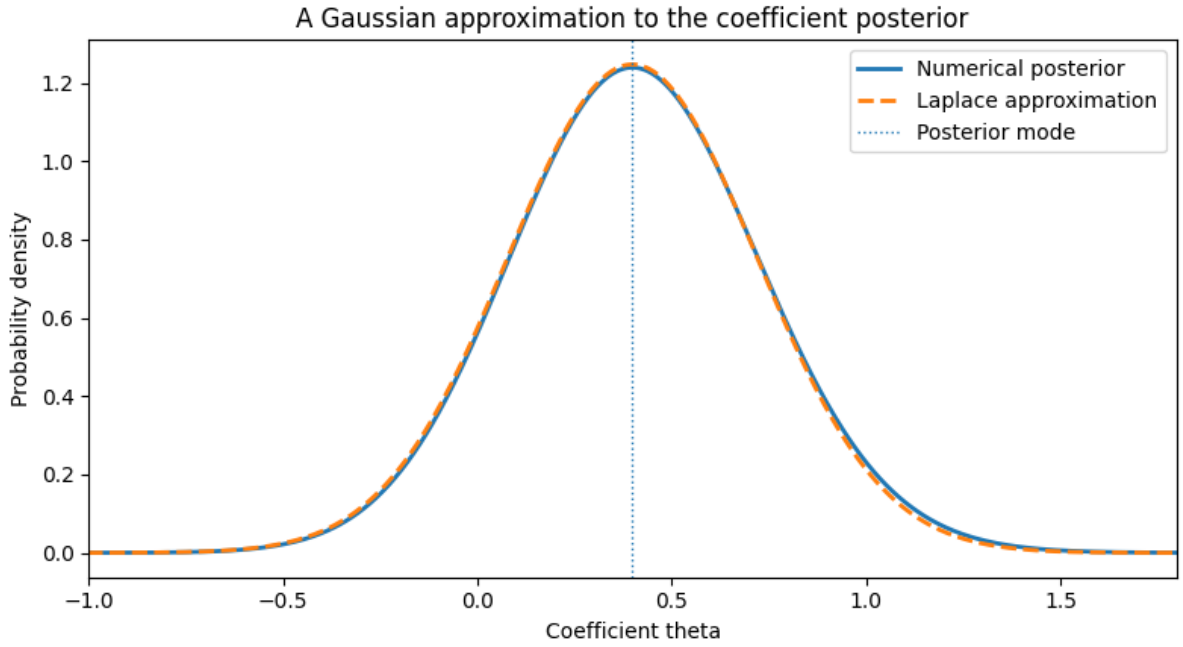


Figure 2: The numerical posterior and the Laplace Gaussian for the complete one-coefficient model. The curves closely agree, with a small difference in shape.

Evaluate $-J$ on a grid, subtract its maximum, exponentiate, and divide by the trapezoidal integral. Subtracting the maximum only rescales the unnormalized density. The normalization removes that scale again. The reference is numerical, not an analytic solution.

Quantity	Numerical posterior	Laplace Gaussian
Mean	0.408884	0.398822
Standard deviation	0.324098	0.319885

The Gaussian mean is the mode, so a small mean difference is expected when the posterior is not perfectly symmetric. This plot supports a good approximation for this example, not a universal guarantee.

Notebook. In Section 4.3, call the same function on the original intercept-and-slope dataset. It returns mean $(-0.389642, 0.961432)^\top$ and marginal standard deviations $(0.530507, 0.377466)^\top$. These define the Gaussian used in the remaining predictive examples.

7. The posterior predictive distribution

Notebook Section 5.1

Say. "We now have a distribution over coefficients. For a new input, each coefficient vector gives a Bernoulli probability. Prediction averages those probabilities over plausible coefficients."

Write and derive

For a single new label,

$$p(y_* \mid x_*, \mathcal{D}) = \int p(y_* \mid x_*, \theta) p(\theta \mid \mathcal{D})\, d\theta.$$

Replacing the coefficient posterior by q gives

$$r_* = \mathbb{E}_q[\sigma(x_*^\top \theta)], \qquad p(y_* \mid x_*, \mathcal{D}) \approx r_*^{y_*}(1 - r_*)^{1-y_*}.$$

The approximate predictive label distribution is Bernoulli. A posterior distribution over the probability itself is a different object.

Under $\theta \sim \mathcal{N}(\theta_{\text{MAP}}, S_n)$, the scalar $a_* = x_*^\top \theta$ is Gaussian. Its moments follow from the linear map:

$$\begin{aligned} m_* &= \mathbb{E}_q[x_*^\top \theta] = x_*^\top \theta_{\text{MAP}}, \\ v_* &= \mathbb{E}_q\left[\{x_*^\top (\theta - \theta_{\text{MAP}})\}^2\right] \\ &= x_*^\top \mathbb{E}_q[(\theta - \theta_{\text{MAP}})(\theta - \theta_{\text{MAP}})^\top] x_* = x_*^\top S_n x_*. \end{aligned}$$

Therefore the d-dimensional integral reduces to

$$r_* = \int_{\mathbb{R}} \sigma(a) \mathcal{N}(a \mid m_*, v_*)\, da.$$

Show why the order of operations matters

For an illustrative distribution assigning equal mass to logits 0 and 4,

$$\sigma(\mathbb{E}[a]) = \sigma(2) \approx 0.8808, \qquad \mathbb{E}[\sigma(a)] = \frac{\sigma(0) + \sigma(4)}{2} \approx 0.7410.$$

This example demonstrates nonlinearity; it is not the Gaussian posterior being fitted. In general, applying the sigmoid at an average coefficient is not Bayesian averaging.

Two quick checks

At $m_* = 0$, symmetry and $\sigma(a) + \sigma(-a) = 1$ give $r_* = 1/2$. At $v_* = 0$, the logit is deterministic and $r_* = \sigma(m_*)$. Both cases are tested in the notebook.

8. Approximating the predictive integral

Notebook Section 5.2 / Gauss–Hermite quadrature

Say. "The projection has reduced prediction to a one-dimensional integral. A deterministic weighted sum can now be used as a reference for Monte Carlo averaging under the same Gaussian."

Write and derive

Write $a = m_* + \sqrt{v_*}\, t$ with $t \sim \mathcal{N}(0, 1)$. Then

$$r_* = \frac{1}{\sqrt{2\pi}} \int_{\mathbb{R}} \sigma(m_* + \sqrt{v_*}\, t) e^{-t^2/2}\, dt.$$

Gauss–Hermite quadrature replaces this weighted integral by

$$r_* \approx \frac{1}{\sqrt{2\pi}} \sum_{j=1}^{J_H} \omega_j \sigma(m_* + \sqrt{v_*}\, t_j).$$

The supplied `roots_hermitenorm` routine returns nodes and weights for $e^{-t^2/2}$, whose weights sum to $\sqrt{2\pi}$. Do not mix it with the other common Hermite convention, which uses e^{-t^2}.

```
nodes, weights = roots_hermitenorm(order)
logits = mean[..., None] + np.sqrt(variance)[..., None] * nodes
result = (sigmoid(logits) * weights).sum(axis=-1) / np.sqrt(2*np.pi)
```

The final axis indexes quadrature nodes; all preceding axes are retained. The function broadcasts input means and variances, and rejects negative variances. At zero variance all nodes have the same logit, so the expression reduces to the sigmoid of the mean.

What is approximate?

There are two separate approximations. First, replace the actual coefficient posterior by a Gaussian. Second, approximate the integral under that Gaussian. Increasing the quadrature order addresses the second approximation only.

The notebook compares 80 and 160 nodes on the main prediction grid. Their largest difference in the executed example is approximately 1.3×10^{-14}. That is a convergence check for this experiment, not a guarantee for all means and variances.

Keep the exercise focused

The quadrature function is supplied. Explain the transformation and weighted sum, then show the checks. Students do not need to derive the quadrature nodes or implement an orthogonal-polynomial algorithm.

Quadrature convention: SciPy documentation [Q].

Exercise 3: Monte Carlo prediction

Notebook Section 5.3 / binary_predictive_mc

Say. "Instead of evaluating a weighted integral, draw plausible coefficients from the Gaussian. Each draw gives a probability curve. Average the curves after applying the sigmoid."

$$\theta^{(s)} \sim \mathcal{N}(\theta_{\text{MAP}}, S_n), \qquad \pi_*^{(s)} = \sigma(x_*^\top \theta^{(s)}), \qquad \widehat{r}_* = \frac{1}{S}\sum_{s=1}^{S} \pi_*^{(s)}.$$

Sampling uses the Gaussian mean and covariance directly:

```
rng = np.random.default_rng(41)
theta_samples = rng.multivariate_normal(theta_map, Sn, size=MC_SAMPLES)
```

Rows of `theta_samples` have shape (S, d). Reuse each row across all test inputs; it represents one complete model. These are independent Gaussian draws, not a Markov chain.

Worked implementation

For `X_star` of shape (n_*, d), the matrix product below has shape (S, n_*):

```
a = theta_samples @ X_star.T
probabilities = sigmoid(a)
mean_probability = probabilities.mean(axis=0)
mc_standard_error = probabilities.std(axis=0, ddof=1) / np.sqrt(len(theta_samples))
return mean_probability, mc_standard_error
```

Conditional on the fitted Gaussian, $\widehat{r}_*$ is unbiased for r_*. Its estimated standard error is

$$\widehat{\text{SE}}_{\text{MC}} = \frac{s_\pi}{\sqrt{S}}, \qquad s_\pi^2 = \frac{1}{S-1}\sum_{s}(\pi_*^{(s)} - \widehat{r}_*)^2.$$

The sample standard deviation measures variability across models. Dividing by $\sqrt{S}$ estimates the numerical error in their mean. There is no need to sample binary labels merely to estimate the average probability.

Checks and hints

Average over the sample axis, not the input axis. Apply the sigmoid before averaging. Identical coefficient draws give identical probability curves and zero estimated Monte Carlo error. One sample is insufficient to estimate a sample standard deviation. The two-logit test catches the wrong order of operations.

Gaussian sampling interface: NumPy documentation [M].

9. Sample curves, then average

Notebook Section 6.1

The three displayed predictions answer different computational questions. MAP plugs in a single coefficient vector. Quadrature and Monte Carlo both average over the same Gaussian coefficient posterior. They should approach each other as integration error decreases, but neither is required to equal the plug-in curve.

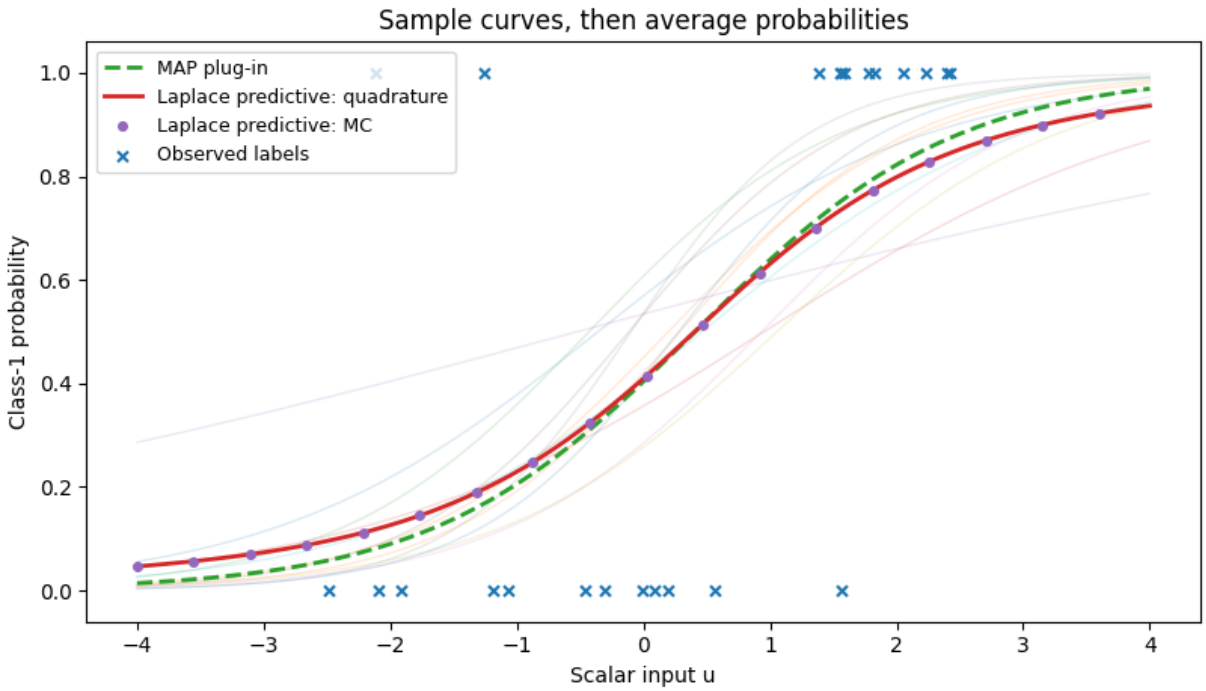


Figure 3: Thin curves are sigmoid responses from sampled coefficients. The quadrature curve and Monte Carlo markers average those probabilities under the same fitted Gaussian.

u_*	MAP	Quadrature	Monte Carlo	MC SE
−3.5531	0.02176	0.05667	0.05611	0.00137
−0.8715	0.22661	0.24818	0.24897	0.00195
1.3631	0.71523	0.70153	0.70096	0.00183
3.5531	0.95375	0.91963	0.91954	0.00146

The main example uses 4,096 coefficient draws. These numbers are outputs from the executed notebook, not targets that a new random run must match exactly.

Questions to ask orally

Which two estimates should agree as the number of draws grows? Which curve would remain unchanged if only the Monte Carlo seed changed? What quantity varies across the thin curves?

Say. "The spread of sampled curves is uncertainty about the probability under the model. The small difference between their estimated mean and quadrature is numerical integration error. These are not the same source of variability."

10. What the shaded interval means

Notebook Section 6.2

Write and derive

The random logit is $a_* \sim \mathcal{N}(m_*, v_*)$ under q. Because the sigmoid is strictly increasing, transforming its Gaussian quantiles gives quantiles of the probability $\pi_* = \sigma(a_*)$:

$$\left[\sigma(m_* + z_{.05}\sqrt{v_*}),\ \sigma(m_* + z_{.95}\sqrt{v_*})\right].$$

Here $z_{.05} \approx -1.64485$ and $z_{.95} \approx 1.64485$. This is a central pointwise 90% interval for the unknown probability under the approximate posterior.

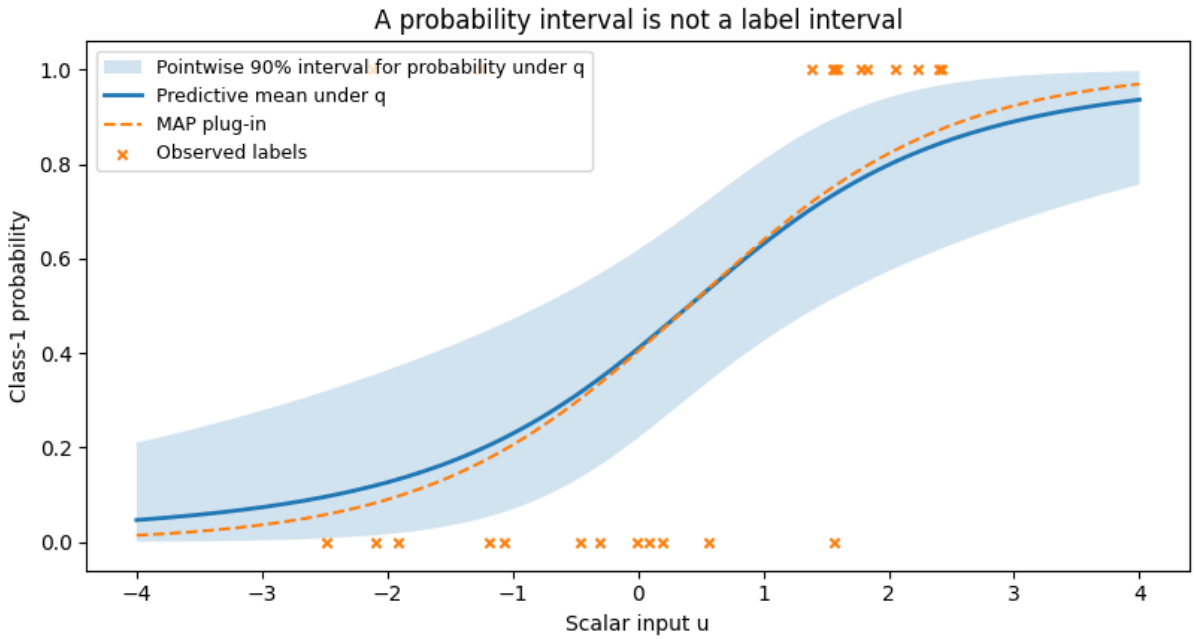


Figure 4: The band represents variation in the class-1 probability at each input. A future binary label still takes only the values zero and one.

Three distinctions to retain

The interval is for a continuous probability, not for the next binary outcome. It is pointwise, not a simultaneous 90% band for every input at once. It is calculated under q, so its interpretation is conditional on the model and the Laplace approximation.

Using more Monte Carlo draws improves a numerical estimate of these quantities; it does not change the probability distribution that defines the band. Here the band is calculated analytically from Gaussian quantiles rather than from noisy sample quantiles.

An oral check

Ask what is random inside the shaded band. The intended answer is the probability induced by uncertain coefficients. Then ask whether drawing more coefficient samples is the same as observing more labels. It is not: one refines computation, the other updates inference.

11. Monte Carlo convergence

Notebook Section 6.3

At $u_* = 1$, the quadrature probability is approximately 0.631122. Compare nested prefixes of one independent Gaussian draw sequence. Increasing the sample count usually reduces typical integration error, but its realized absolute error need not decrease monotonically.

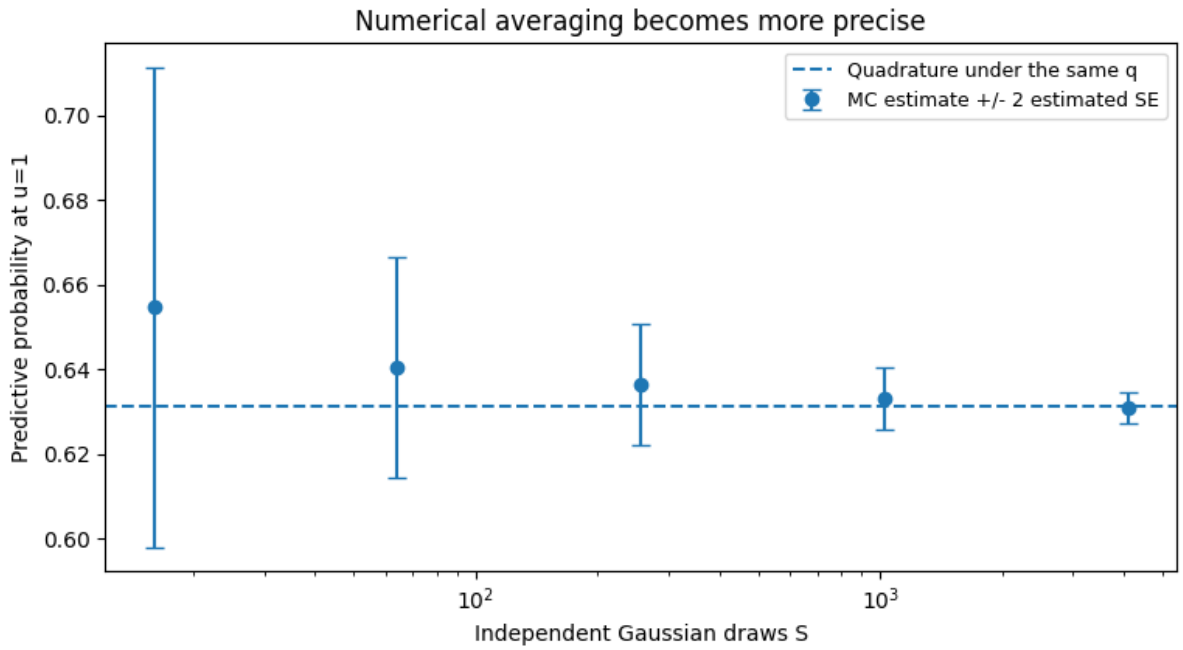


Figure 5: The error bars are two estimated Monte Carlo standard errors around the estimated average probability. The horizontal line is quadrature under the same Gaussian.

Draws S	MC probability	Estimated SE	Absolute error
16	0.654682	0.028364	0.023560
64	0.640307	0.013059	0.009185
256	0.636417	0.007097	0.005295
1,024	0.633255	0.003668	0.002133
4,096	0.630768	0.001824	0.000354

The theoretical standard deviation of the sample average is $\sqrt{\mathrm{Var}_q(\pi_*)/S}$. Therefore multiplying S by four halves that theoretical scale. The estimated standard error fluctuates because the sample variance also fluctuates.

Say. "The Gaussian posterior is fixed throughout this calculation. More draws do not make the posterior narrower, learn a better slope, or correct the Laplace approximation. They give a more accurate average under the distribution we already fitted."

If time is tight, show only the first and last rows. Keep the distinction between numerical error and posterior uncertainty; skip discussing the plotting details or every intermediate budget.

12. A weakly constrained linear slope

Notebook Section 6.4 / uncertainty example

Use a new toy dataset to make uncertainty about coefficients visible. The model is still linear in one scalar input:

$$p(y = 1 \mid u, \theta) = \sigma(\theta_0 + u\theta_1).$$

Initially, the 24 inputs lie in $[-0.15, 0.15]$. The prior standard deviations are 2.5 for the intercept and 3 for the slope; both prior means are zero. The generating coefficients are $(0, 1.5)^\top$ and are used only to simulate labels and draw a reference curve.

Near $u = 0$, changes in the slope have little effect on the logit. The observations therefore constrain the intercept more strongly than the slope. Under the fitted Gaussian,

$$\mathbb{E}_q[\theta_1] \approx 1.46835, \qquad \mathrm{SD}_q(\theta_1) \approx 2.50281, \qquad \Pr_q(\theta_1 > 0) \approx 0.72129.$$

There remains substantial probability on slopes with the opposite sign. This is a case in which a linear model can express strong epistemic uncertainty outside the observed input interval.

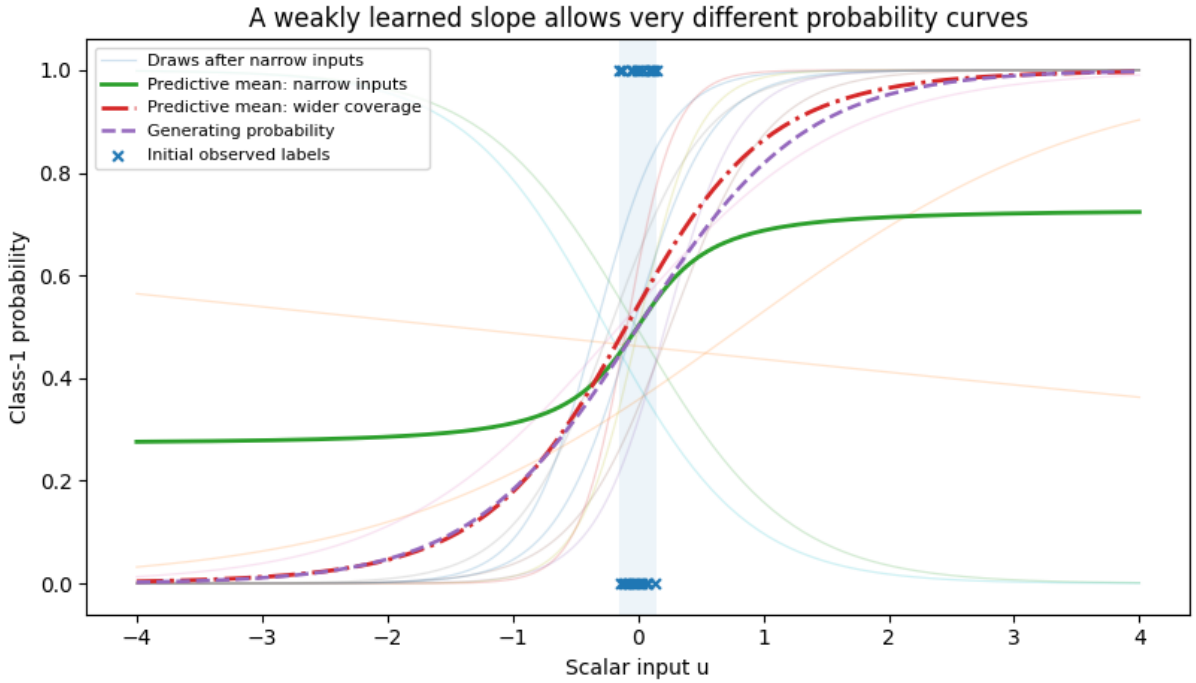


Figure 6: Sampled models fitted to narrow-range inputs disagree strongly outside that range. The wider-coverage predictive mean comes from the refit discussed on p. 20.

Say. "The likelihood can be informative about one coefficient and weakly informative about another. A single fitted curve hides that distinction. The sampled curves expose what the observations have not yet determined."

Ask why the models agree more near zero. Keep the discussion tied to the observed design and fitted posterior, rather than claiming that any linear model must become uncertain far from its data.

13. Estimate the uncertainty components

Notebook Section 6.4 / use the same probability draws

For each input, let $\pi_*^{(s)} = \sigma(x_*^\top \theta^{(s)})$ and $\widehat{r}_* = S^{-1} \sum_s \pi_*^{(s)}$. The notebook computes variance-based quantities:

$$\widehat{U}_{\text{total}} = \widehat{r}_*(1 - \widehat{r}_*),$$
$$\widehat{U}_{\text{aleatoric}} = \frac{1}{S} \sum_s \pi_*^{(s)}(1 - \pi_*^{(s)}),$$
$$\widehat{U}_{\text{epistemic}} = \frac{1}{S} \sum_s (\pi_*^{(s)} - \widehat{r}_*)^2.$$

These obey $\widehat{U}_{\text{total}} = \widehat{U}_{\text{aleatoric}} + \widehat{U}_{\text{epistemic}}$. To see the identity, set $\widehat{m}_2 = S^{-1} \sum_s (\pi_*^{(s)})^2$. The two components are $\widehat{r}_* - \widehat{m}_2$ and $\widehat{m}_2 - \widehat{r}_*^2$, which sum to $\widehat{r}_*(1 - \widehat{r}_*)$.

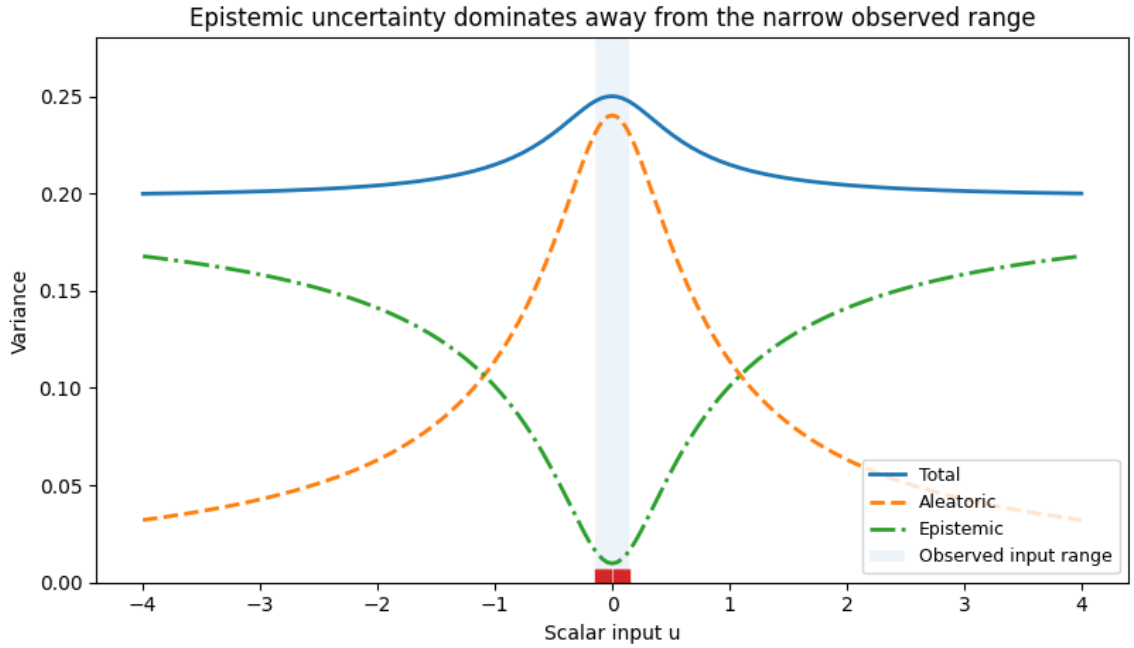


Figure 7: Near the observations, conditional label variability dominates. Away from the narrow range, disagreement between sampled classifiers contributes most of the total variance in this example.

```
mean_probability = probability_samples.mean(axis=0)
total = mean_probability * (1-mean_probability)
aleatoric = (probability_samples * (1-probability_samples)).mean(axis=0)
epistemic = probability_samples.var(axis=0, ddof=0)
```

Use `ddof=0` here so the finite-sample identity holds exactly up to round-off; `ddof=1` remains appropriate for estimating the Monte Carlo standard error. These are variances, not entropies. They describe uncertainty within the specified model and approximate posterior, not every possible source of model error.

14. Add observations and refit

Notebook Section 6.4 / wider input coverage

Add 60 observations with inputs spanning $[-3, 3]$, retaining the original 24. Refit the same model with the same prior. Both fits use the same Monte Carlo sample count; only the observed dataset changes.

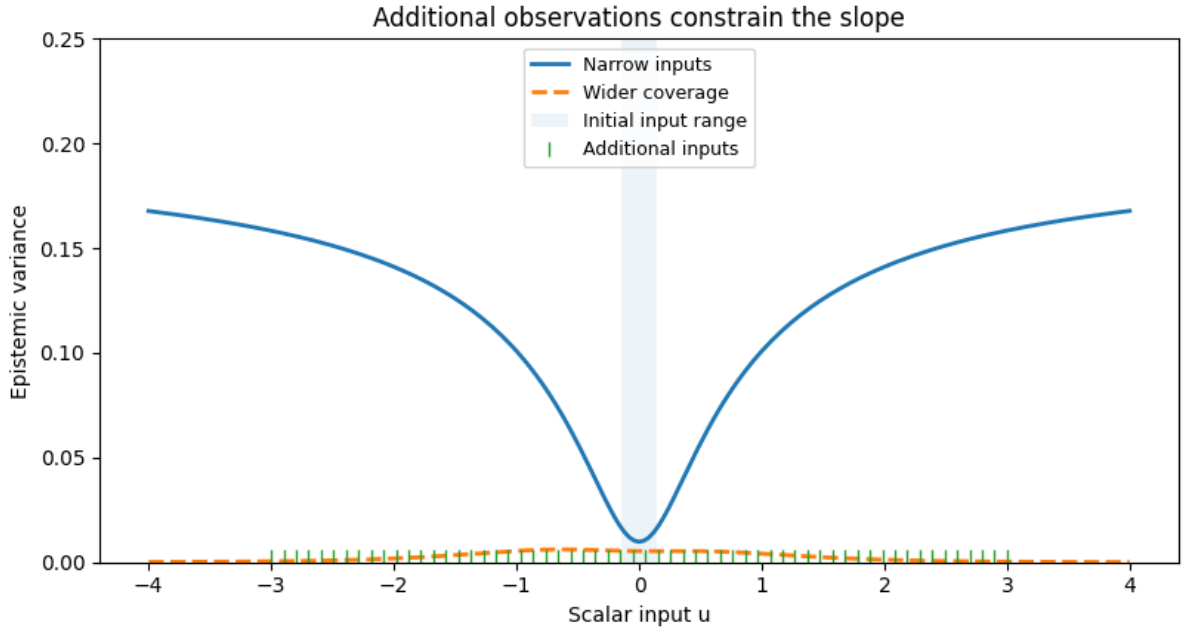


Figure 8: Additional observations over a wider input range substantially reduce epistemic variance in this simulated example. The markers on the axis indicate the added inputs.

Quantity	Narrow inputs	Wider coverage
Number of labels	24	84
MAP slope	1.46835	1.76994
Slope SD under q	2.50281	0.43258
$\Pr_q(\theta_1 > 0)$	0.72129	0.99998
Total variance at $u = 3$	0.201233	0.009530
Aleatoric component at $u = 3$	0.042810	0.009219
Epistemic component at $u = 3$	0.158423	0.000310

Say. "More Gaussian draws improve an integral estimate. More observations can change the posterior itself. Here, labels at inputs with larger magnitudes supply information about the slope, so the fitted Gaussian no longer places substantial mass on negative slopes."

The conditional-variance component can also change after refitting because it is averaged over a changed posterior. Do not claim it must stay fixed. Likewise, posterior variance need not decrease at every input after every realized dataset update. The displayed reduction is the result of this particular simulation.

Close with a discussion of what was learned from the additional observations, rather than a general claim about OOD detection or calibration. Those topics remain in the appendices.

15. Close the inference chain

Oral synthesis / no written-response exercise

Leave these equations on the board:

$$p_i = \sigma(x_i^\top \theta), \qquad \theta \sim \mathcal{N}(m_0, S_0),$$

$$\theta_{\text{MAP}} = \arg\min_\theta J(\theta), \qquad S_n = [\nabla^2 J(\theta_{\text{MAP}})]^{-1}, \qquad q(\theta) = \mathcal{N}(\theta_{\text{MAP}}, S_n),$$

$$r_* = \int \sigma(a)\mathcal{N}(a \mid x_*^\top \theta_{\text{MAP}}, x_*^\top S_n x_*)\, da \approx \frac{1}{S}\sum_s \sigma(x_*^\top \theta^{(s)}).$$

The approximate predictive distribution of one future label is Bernoulli(r_*).

What Exercise 2 accomplished

Students fitted a mode, evaluated its Hessian, and constructed a Gaussian approximation. The one-coefficient density plot made the approximation directly visible. The same function then returned a two-coefficient Gaussian for all subsequent predictive calculations.

Questions and concise answers

Why invert the Hessian? A Gaussian negative log density contains the inverse covariance in its quadratic term. Matching that quadratic with the posterior expansion gives $S_n = H_{\text{MAP}}^{-1}$. In one dimension, variance is simply the reciprocal of the second derivative.

Does the Gaussian have the exact posterior mean? Its mean is the posterior mode. The scalar comparison showed that the numerical posterior mean can differ slightly.

Why not use the sigmoid of the MAP? That ignores variation of the probability over plausible coefficients. Averaging after a nonlinear transformation is generally different from transforming an average.

Will more Monte Carlo draws fix a poor Gaussian approximation? No. They reduce integration error for that Gaussian, not the discrepancy between it and the actual posterior.

Does an uncertain label imply an uncertain model? No. A probability near one-half can remain even when sampled models largely agree. The variance components in the final example separate conditional Bernoulli variation from disagreement between model probabilities.

The final sentence

"We have moved from one fitted classifier to a distribution of plausible classifiers, and from that distribution to a prediction. Laplace made the posterior manageable; quadrature and Monte Carlo turned it into probabilities we can use."

Board formula sheet

Keep this page available while teaching

Model and objective

$$a = X\theta, \quad p = \sigma(a), \quad \sigma(a) = \frac{1}{1+e^{-a}}, \quad \sigma'(a) = \sigma(a)[1-\sigma(a)].$$

$$\ell_{\text{lik}} = \sum_i [y_i \log \sigma(a_i) + (1-y_i)\log \sigma(-a_i)].$$

$$\theta \sim \mathcal{N}(m_0, S_0), \quad P_0 = S_0^{-1}, \quad J = -\ell_{\text{lik}} + \tfrac{1}{2}(\theta - m_0)^\top P_0(\theta - m_0).$$

$$g = X^\top(p-y) + P_0(\theta - m_0), \qquad H = X^\top W X + P_0, \qquad W = \operatorname{diag}(p_i(1-p_i)).$$

Newton and Laplace

$$H\delta = -g, \qquad \theta_{\text{new}} = \theta + \eta\delta, \qquad \theta_{\text{MAP}} = \arg\min J.$$

$$J(\theta) \approx J(\theta_{\text{MAP}}) + \tfrac{1}{2}(\theta - \theta_{\text{MAP}})^\top H_{\text{MAP}}(\theta - \theta_{\text{MAP}}),$$

$$H_{\text{MAP}} = \nabla^2 J(\theta_{\text{MAP}}), \quad S_n = H_{\text{MAP}}^{-1}, \quad q(\theta) = \mathcal{N}(\theta_{\text{MAP}}, S_n).$$

$$\text{One coefficient:} \qquad q(\theta) = \mathcal{N}\left(\theta_{\text{MAP}}, \frac{1}{J''(\theta_{\text{MAP}})}\right).$$

Prediction and integration

$$m_* = x_*^\top \theta_{\text{MAP}}, \quad v_* = x_*^\top S_n x_*, \quad r_* = \mathbb{E}_{a\sim\mathcal{N}(m_*, v_*)}[\sigma(a)].$$

$$r_* \approx \frac{1}{\sqrt{2\pi}} \sum_j \omega_j \sigma(m_* + \sqrt{v_*}\, t_j), \quad \text{Hermite weight } e^{-t^2/2}.$$

$$\theta^{(s)} \sim \mathcal{N}(\theta_{\text{MAP}}, S_n), \quad \pi_*^{(s)} = \sigma(x_*^\top \theta^{(s)}), \quad \widehat{r}_* = \frac{1}{S}\sum_s \pi_*^{(s)}, \quad \widehat{\text{SE}}_{\text{MC}} = \frac{s_\pi}{\sqrt{S}}.$$

$$\text{Pointwise 90\% probability interval:} \quad [\sigma(m_* - 1.64485\sqrt{v_*}),\ \sigma(m_* + 1.64485\sqrt{v_*})].$$

$$U_{\text{total}} = r_*(1-r_*) = \mathbb{E}_q[\pi_*(1-\pi_*)] + \text{Var}_q(\pi_*).$$

Use the summed-objective Hessian. Average after the sigmoid. Probability spread, future-label variation, and numerical integration error are different quantities.

Appendix navigation

Optional reading and experiments / not part of the timed lesson

The notebook appendices remain available, but `RUN_APPENDICES=False` prevents their computations from running by default. The main lesson does not require the image data or the additional exercise.

Appendix	Contents
A	Numerical Newton step, complete separation, and prior sensitivity.
B	General Laplace normalization, evidence, objective scaling, and a two-dimensional numerical posterior audit.
C	Covariance correlations, diagonal approximations, frozen image features, the softmax last-layer model, and coding Exercise 4.
D	Scoring rules, reliability diagrams, calibration diagnostics, OOD confidence, and ROC evaluation.
E	Analytic logistic–Gaussian shortcut and a warning about extrapolation limits.
F	Gaussian maximum entropy under moment constraints.
G	Pretrained-feature interfaces and a repeated-Monte-Carlo diagnostic.

Where numerical linear algebra belongs

The main exercise explicitly inverts a one- or two-dimensional Hessian. For larger problems it can be preferable to solve linear systems instead of forming a dense inverse. The optional routines use Cholesky factors to solve systems, compute log determinants, or sample from an inverse covariance. These are numerical implementations of the Gaussian calculation, not additional assumptions required to derive Laplace.

The word precision in the appendices means inverse covariance. The notation there is $A = H_{\mathrm{MAP}}$ and $S_n = A^{-1}$. Neither the terminology nor the factorization algorithm is needed to understand the main density comparison.

Softmax is only a multiclass extension

The main model and every binary experiment use the sigmoid. The multiclass appendix still uses softmax, because it models several mutually exclusive labels rather than one Bernoulli label. It is a separate extension, not the response function used to fit the main logistic regression model.

What the scalar density plot does not replace

The one-coefficient plot shows a complete, easily understood posterior. The two-dimensional grid in Appendix B provides a more detailed audit of the original intercept-and-slope fit. Do not transfer numerical accuracy claims from one dataset to the other without that audit.

Sources and numerical checks

Reference / reproducibility

Material accompanying this guide

[N] *Bayesian logistic regression with the Laplace approximation*, the accompanying student and solutions notebooks. Main Sections 1–6 provide the displayed model, three coding exercises, examples, and numerical results. Appendix references remain in the notebooks.

[I] *Bayesian Logistic Regression with the Laplace Approximation*, supplied IPPI seminar notes. The sigmoid likelihood, Gaussian-prior formulation, Laplace expansion, and predictive integral are developed there in Sections 2–6. This guide uses the negative-log-posterior convention throughout.

The one-coefficient demonstration is specified by 40 labels, 24 positive, and a $\mathcal{N}(0, 2.5^2)$ prior. Its numerical posterior is evaluated on 2,001 nodes over $[-4, 4]$. This is distinct from the 24-observation, two-coefficient simulated dataset used for the main predictions.

Implementation references

[S] SciPy, `expit` and `log_expit`: sigmoid probabilities and directly evaluated log probabilities.

[Q] SciPy, `roots_hermitenorm`: quadrature for the $e^{-t^2/2}$ weight.

[M] NumPy, `Generator.multivariate_normal`: Gaussian draws specified by a mean and covariance. Floating-point and Monte Carlo outputs can differ slightly across numerical-library builds; distributional checks matter more than bitwise matching.

Checks used for this edition

The notebook tests check analytic derivatives against finite differences, finite loss values at extreme logits, Newton convergence, a scalar Laplace case with known mode and variance, predictive averaging and sample axes, quadrature limiting cases, and the finite-sample uncertainty identity. The completed and unfinished student paths and enabled appendices are executed separately.

The scalar numerical reference was also checked on a wider interval and a finer grid; its mean and standard deviation are stable at the displayed precision. The original two-coefficient MAP and covariance agree with the previous formulation within numerical tolerance. The Gaussian sampling interface changed, so individual Monte Carlo draws and associated sample tables have been regenerated.

Exact package versions and execution records are included in the accompanying archive. Requirements for the main lesson and optional appendices are listed separately.